# OPTT Analysis

Oliver Reece

## Abstract

Founded in 1894, Ocean Power Technologies (OPTT) is a martime-focused company with an emphasis on finding solutions to the need for more efficient/productive infrastructure. Their services span defense and security, oil and gas procurement, and off-shore wind markets.

OPTT went public on the NASDAQ on the 25th of April, 2007. Initially, the stock exploded, with a closing price high of $3,652.00 on November 12th, 2007. Ever since, the stock has been on the decline, falling under $5 per share in early 2019. Crossing that $5 threshold means the stock is now classified as a penny stock. Ocean Power Technologies hosts a market cap of $74.6 Million (at the time of writing), thus designating the company as a micro-cap company.

Since approximately 2012, OPTT has suffered in many ways that have led to investor distrust. The three main areas of concern are the high risk designation due to high volatility, share dilution, and multi-million dollar operating losses. For example, between 2012 and 2026, the number of shares (approximately) increased from 5 million to 180 million while total operating losses during that time exceeded $300 Million.

That being said, OPTT is not entirely out of the market. On January 6th, 2026 OPTT announced they had been awarded a contract by the U.S. Department of Homeland Security for the purpose of supplying the U.S. Coast Guard with their new PowerBuoy systems. The contract was valued at $6.5 Million. This contract was not only a lifeline for OPTT but marked the transition from a sustainability focused private company into what is now essentially a revenue focused defense contractor.

In this project, I use statistical modelling and financial metrics to analyse OPTT's performance and risk overtime. From there I aim to make inferences about whether thus far the contract has begun to aid OPTT recover. I then test my inferences using machine learning algorithms to predict future stock prices.

I will be using OLS regression models for performance analysis, using the log of Volume as the dependent variable. I am using volume because in contrast to the other metrics I can choose from, Volume is able to differentiate between direction and attention in the market. Volume focuses on how many shares are bought or sold, meaning it is a good measure of general activity

surrounding the stock itself as opposed to what the public think of the stock. I take the log of Volume due to it being right-skewed with a median of 7,710 vs a mean of 1,974,392.

I will use a logistic regression model for risk analysis, using a "risk event" binary as the dependent variable. The risk event will be triggered when a significant price drop or sell-side volume spike occurs, or both. The models will be ordinal in nature, each with 3 "severity" tiers, defined as:

- **Level 1** -> No Risk Event: Price Drop $< 3\%$ and Volume Multiple $< 1.25$
- **Level 2** -> Moderate Risk Event: Price Drop between 3% - 7% (inclusive) or Volume Multiple between 1.25 - 2 (inclusive) AND neither metric surpasses Level 3 thresholds.
- **Level 3** -> High Risk Event: Price Drop $> 7\%$ or Volume Multiple $> 2$

I will then move into predictive modelling, using Random Forest, XGBoost, and LSTM Networks to predict future performance.

For the LSTM Network, the adjusted closing price will be the outcome variable. I am using adjusted closing over opening or closing price due to the notion that an ML algorithm will look at fluctuations between day as catastrophic economic events if the drop is significant enough from one day to another. It is also of note that if that drop is caused by a volatility spike, the adjusted closing price will catch it. Moreover, the adjusted closing price would back-trace historical data which would prevent the model from falling for false inter-day price gaps.

For the Random Forest and XGBoost algorithms, it does not make sense to predict price as a raw dollar metric due to the lack of trend-awareness that these algorithms possess. Both algorithms function by breaking the data down into rectangular cross sections based on real historic data. Thus, these models will never predict a future dollar price that exceeds the upper and lower bounds of the data - a reality that does not exist in the stock market. As a solution, I will predict returns using a calculated percent change metric as the outcome variable. This percent change will be calculated using adjusted closing price

To conclude, I will compare the effectiveness of the 3 ML models using error scores to determine which stand-alone algorithm performed best.

Why does any of this matter? Firstly, I enjoy machine learning and investing, so this was a natural project for me to take on. Secondly, it is an interesting question to ask - can traditional ML handle the noise of penny stocks? Currently, analysts rely on hybrid algorithms that incorporate classification models *and* sentiment analysis. In most cases, LSTMs tend to confuse the noise of unexpected price spikes (that are overly common in penny stocks) for true trends, and I want to see the comparison myself.

## The Data

In order to take on this project, I utilise a variety of methods to procure data. The main pathway is the `tidyquant` package from which I am able to download historic data about the desired ticker. `tidyquant` primarily leans on Yahoo Finance for key metrics, but also pulls from several other databases such as the Federal Reserve Economic Data (FRED).

I also make queries to NewsAPI and Finnhub, two APIs that allow me to scrape for media related to my chosen ticker in order to perform sentiment analysis in conjunction with the machine learning models. Unfortunately, there is limited results for Ocean Power Technologies and as such I do not continue down this path.

Using `tidyquant`, I retrieve data from April 25th, 2007 (IPO) until June 21st, 2026 which is the date of writing, although the data becomes usable after row 34 due to the staggered system in which some calculated metrics such as MACD work. That being said, the complete dataset boasts 4,819 observations across 17 variables (22 when including ticker name, year, month, month name, and day).

Of these 17 variables, some notable measures include:

- **`open_price`**: Stock price at open on a given trading day
- **`close_price`**: Stock price at close on a given trading day
- **`Volume`**: Number of shares traded (bought or sold) during a given trading day
- **`true_range`**: Total price volatility by finding the greatest distance between current high, current low and previous closing price
- **`ATR`**: Moving average of true range over a 14-day period
- **`rsi`**: Relative Strength Index: Measures magnitude and speed of price fluctuations on a scale of 0-100 (momentum indicator)
- **`macd_hist`**: Measures if the stock's momentum is speeding up or slowing down
- **`obv`**: On-Balance Volume: Relationship metric between volume and price used to spot divergences

To illustrate the structured dataset, rows 34–39 are displayed below. Because several indicators rely on rolling or exponential moving-average calculations of varying lengths (e.g., a 14-day window for ATR and RSI, and a 26-day EMA - Exponential Moving Average - for the MACD trendline, followed by a further 9-day EMA for the MACD signal line), the initial rows of the dataset default to NA values until each indicator has accumulated sufficient historical data to produce a valid output. The MACD signal line, in particular, does not return numeric values until row 34, as it requires the MACD line itself to first be calculated. Rows 34–39 are therefore displayed to ensure all indicators, including MACD and its derived histogram, are represented with complete, numeric, and non-NA values.

In short, the following table represents values from June 12th, 2007 - June 19th, 2007. June 16 and June 17 are omitted as they fall on a Saturday and Sunday which are non-trading days, meaning no data can be collected on those days.

| open_price | close_price | Volume | true_range | ATR | rsi | macd_hist | obv |
|---|---|---|---|---|---|---|---|
| 3036 | 3024 | 386 | 150 | 135.4286 | 40.01469 | 0.0571432 | 7232 |
| 3004 | 3128 | 186 | 132 | 134.0000 | 45.91322 | 0.3497787 | 7418 |
| 3144 | 3182 | 286 | 128 | 128.2857 | 48.73218 | 0.6559246 | 7704 |
| 3190 | 3222 | 236 | 80 | 126.2857 | 50.77863 | 0.9234428 | 7940 |
| 3240 | 3154 | 120 | 88 | 117.5714 | 47.32051 | 0.9297880 | 7820 |
| 3160 | 3284 | 324 | 246 | 128.0000 | 53.79843 | 1.1732797 | 8144 |

## MACD Explained

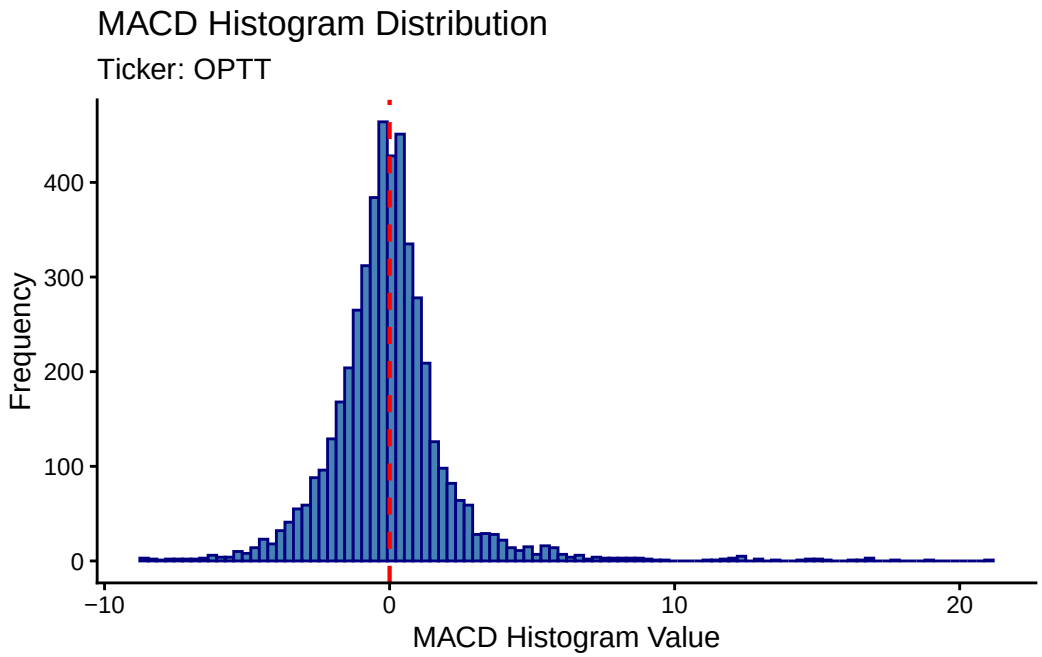
MACD is the Moving Average Convergence Divergence, essentially a tool used to determine buy/sell signals, price trends, and momentum. There are 3 main parts to MACD: the trendline, the signal line, and the histogram.

The MACD trendline is the main part of this indicator. It is calculated by subtracting a 26-period EMA from a 12-period EMA. The 26-period EMA is the longer-term trend and the 12-period EMA is the shorter-term momentum. EMA stands for Exponential Moving Average and is a metric that smooths out the price data to gauge a better view of the direction and momentum of trends. It is used over standard moving averages because it puts higher emphasis on later data points, which matter most in the stock market. Due to the volatility of the OPTT stock, I elect to use 12 and 26 period EMAs as opposed to 50 and 200 period EMAs which might be more favourable to longer term analysis or more stable stocks.

The MACD signal line is a 9-period moving average of the MACD trendline. In other words, it acts as a "slower" baseline metric for the momentum of the stock which can then be compared to the "faster" trendline.

That relationship is expressed by the MACD histogram. It is a metric that helps investors determine it is a bullish vs bearish market, which in turn aids in determining whether or not to buy or sell the stock. I calculate this value by subtracting the MACD signal from the MACD trendline value for any given observation, represented as `macd_hist` in the table above.

Below are the values of the MACD histogram displayed as a histogram:




Looking at the pictured histogram, we see that the data is (for the most part), normally distributed around 0 with a slight right-skew. This tells me that on average, across all time, momentum is neutral, meaning no real sustained bullish nor bearish trends. The extended tail that causes the right-skew is most likely a result of the initial bullish trend when the company first became public on the NASDAQ. This would explain why we don't see a tail of equal length on the left: the decline was slower, across a longer period of time, meaning the fluctuations during the decline will mitigate the effect of that sustained decline. On a similar stream, we would expect the MACD histogram of a higher valued, more stable stock to have a tighter distribution. For example, an S&P500 tracker like VOO could be expected to have a tighter distribution with a slight right-shift (not skew) due to it's sustained upward trend without the extreme volatility witnessed in OPTT.

## Sentiment Analysis

```
#finnhub Sentiment News Scoring

#connect to finnhub
finnhub_url <- paste0(
  "https://finnhub.io/api/v1/company-news?symbol=OPTT&from=2026-05-21&to=2026-
06-21&token=", finnhub_key)

#process data using httr and jsonlite
```

```
finnhub_raw_data <- GET(finnhub_url)
finnhub_parsed <- fromJSON(content(finnhub_raw_data, "text"))

#preview parsed data
head(finnhub_parsed)
str(finnhub_parsed)

#clean parsed data into a usable format
finnhub_tidy <- finnhub_parsed |>
  mutate(
    Date = as_date(as_datetime(datetime)),
    Text = headline
  ) |>
  rename(Source = source) |>
  select(Date, Text, Source, url)

#only 1 article


# News API Sentiment Scoring

#connect to NEWSAPI using newsanchor
news_api_result <- get_everything(
  query = '"Ocean Power Technologies" OR "OPTT stock" OR "NASDAQ:OPTT"',
  from = "2026-08-12",
  to = "2026-09-09",
  api_key = news_api_key
)

#check to see if we got data
news_api_result$metadata$total_results
news_api_result$metadata$status_code
news_api_result$results_df

#3 articles

#clean news api data
news_tidy <- news_api_result$results_df |>
  mutate(
    #change column names to match finnhub to make joining easier
    Date = as_date(published_at),
    Text = title
  ) |>
```

```
  rename(Source = name) |>
  select(Date, Text, Source, url)


#create and display combined scraped data
scraped <- full_join(finnhub_tidy, news_tidy, by = c("Date", "Text", "Source", "url"))
```

After querying both NewsAPI and Finnhub, across many trial runs between June and August 2026, on average 6 articles were returned, primarily originating from sources of high bias. Due to concerns of limited data and subsequent biases, I will not be using sentiment analysis scores within my models. The table below shows the date and source of each relevant article.

```
#| output: true
scraped |>
  select(Date, Source) |>
  kable(caption = "Sentiment Data Article Origin") |>
  kable_styling(font_size = 10)
```

These articles are not sufficient in terms of volume, however if used, another bias concern would arise: on average 43.75% of represented information would stem from the same source - GlobeNewswire. GlobeNewswire is not an independent journalism company, rather the company is paid by other companies (like Ocean Power Technologies) to publish and distribute press releases. In other words, the publications represented in this scenario were written by Ocean Power Technologies themselves.

Instead of sentiment data, the following analysis will focus on price-derived technical indicators alongside trading volume. These variables are derived directly from empirical trading data, avoiding the sample-size and bias concerns associated with the limited sentiment data.

In order to maintain structural robustness via alternative data streams, the analysis will pivot to integrate macroeconomic corporate health metrics, specifically outstanding share dilution and order backlog growth.

## Alternate Data Procurement

To supplement the market data, I require fundamental financial metrics to accurately gauge the internal health of the company. Specifically, I focus on Outstanding Shares, the Backlog-Sales Ratio, and Operating Loss. Outstanding Shares and Operating Loss are sourced from the balance sheet and income statement, respectively. I aimed to utilise the Alpha Vantage API integrated within the `tidyquant` package however due to a lack of available information, I instead scraped the information from the SEC filings. I did so using Google's NotebookLM.

The Backlog-Sales Ratio, calculated as Total Backlog Value divided by Total Revenue, requires historical backlog data sourced directly from SEC filings - I used the same Notebook to accomplish this.

Upon reviewing these SEC filings, a notable anomaly emerged: order backlog was not reported between 2015 and 2020. This gap coincides with OPTT's fundamental restructuring phase, during which the company transitioned from grid-dependent buoys to the self-sufficient models they manufacture today. Essentially, these were highly concentrated research and development (R&D) years. Under SEC regulations up until 2020, if a company's order backlog became so insufficient that it lacked material impact on their overall finances—effectively nearing $0 - the company was permitted to omit the data entirely. To accurately reflect this operational reality, I impute the missing backlog data during these years with values of $0.

Because the fundamental financial data is reported quarterly, it must be disaggregated into monthly intervals to align with the broader dataset. Linear interpolation was immediately ruled out; because it connects quarterly data points using trendlines, it inherently relies on future data, which would introduce severe look-ahead bias into the historical analysis.

A standard alternative is the Last Observation Carried Forward (LOCF) method, a forward stepwise function that simply holds the quarterly value constant across the subsequent months. By filling in the gaps with known data, we aim to limit data-sparsity bias. However, the LOCF method introduces its own set of issues such as information bias and volatility attenuation bias. By assuming the metric remains constant between quarters, the LOCF function creates artificial steps in the data, ultimately reducing its natural variance which is especially important to account for given OPTT's status as a penny stock. Consequently, models trained on this data might produce artificially stable results, leading to dangerous, false "low risk" predictions.

To address this quarterly-to-monthly discrepancy, I evaluated three primary pathways:

- **No Imputation:** Leaving the data blank would result in statistically weak and highly fragmented analyses.
- **LOCF Imputation:** Utilising the stepwise function introduces the aforementioned volatility attenuation and false-stability biases.
- **Averaging Existing Metrics:** Leaving the gaps empty but averaging other higher-frequency metrics across quarters introduces significant Information Loss Bias and Temporal Masking.

Ultimately, I opted against LOCF and standard interpolation in favour of advanced temporal disaggregation techniques. Specifically, I rely on the Denton-Cholette and Chow-Lin models. The Denton-Cholette method forces the newly generated monthly data to track the volatility of a correlated, higher-frequency tertiary metric. Similarly, the Chow-Lin method utilises a regression-based approach to achieve this alignment. By leveraging these temporal disaggregation techniques, I can successfully convert the quarterly data into monthly intervals while preserving historical volatility and strictly avoiding look-ahead bias.

First I will use the Chow-Lin regression based method. This is more consistent with standard econometric modelling practices, which makes more sense in this scenario.

```
backlog <- read.csv("order_backlog.csv")
shares_loss <- read.csv("OPTT_Data.csv")
```

backlog data introduces bias. replacement needed.

```
alt_data <- shares_loss |>
  rename(Year = Fiscal.Year, shares_outstanding = Outstanding.Shares, operating_loss = Operat
  mutate(Year = rev(Year),
         shares_outstanding = rev(shares_outstanding),
         operating_loss = rev(operating_loss)
         ) |>
  mutate(operating_loss = ifelse(Year == 2023, -29403000, operating_loss))

head(alt_data)



#join alt and normal data into "optt_model dataframe"

#optt_model <- alt_data + normal data (alt_data join optt_tidy)
optt_model <- optt_tidy |> left_join(alt_data)
#optt_model_tidy <- mutate()()()
    #remove NAs --> row 33 is last obs w/ NA (macd_hist, macd_signal)
    #temporal disagg for outstanding shares, operating loss --> chow lin should suffice
```

## Ordinary Linear Regression Model

```
ols <- lm(log(Volume) ~ rsi + macd_hist + shares_outstanding + operating_loss,
          data = optt_model_tidy)
summary(ols)

#check residuals
plot(ols)
```

## Logistic Regression Model

```
logit <- glm(log(Volume) ~ rsi + macd_hist + shares_outstanding + operating_loss,
             family = binomial,
             data = optt_model_tidy)
```

## Forecasting/Prediction ML Models

```
#going to measure price by close price, so that is my metric of choice to split by
# split data for training and testing
#70% train, 30% test

set.seed(10092026)
train_index <- createDataPartition(optt_model_tidy$close_price, p = 0.7, list = FALSE)

Train <- optt_model_tidy[train_index, ]
Test <- optt_model_tidy[-train_index, ]
```

### Random Forest

```
install.packages("randomForest")
library(randomForest)
-bootstrap code-
-use complexity parameters: min_samples_split, max_depth, n_trees (10k)-
```

### XGBoost

```
install.packages("xgboost")
library(xgboost)

labels <- as.integer(iris$Species) - 1
features_matrix <- as.matrix(iris[, 1:4])
dtrain <- xgb.DMatrix(data = features_matrix, label = labels)
```

### LSTM Network

```
install.packages("keras")
install.packages("tensorflow")
keras::install_keras()
```

```
library(keras)
library(tensorflow)
```

## References


Chow-Lin Temporal Disaggregation: Chow, G. C., & Lin, A. L. (1971). Best Linear Unbiased Interpolation, Distribution, and Extrapolation of Time Series by Related Series. The Review of Economics and Statistics, 53(4), 372-375.

Denton-Cholette Temporal Disaggregation: Denton, F. T. (1971). Adjustment of Monthly or Quarterly Series to Annual Totals: An Approach Based on Quadratic Minimization. Journal of the American Statistical Association, 66(333), 99-102.

dplyr package: Wickham, H., François, R., Bryan, J., & Müller, K. (2023). dplyr: A Grammar of Data Manipulation. R package version 1.1.4. https://CRAN.R-project.org/package=dplyr

ggplot2 package: Wickham, H. (2016). ggplot2: Elegant Graphics for Data Analysis. Springer-Verlag New York. https://ggplot2.tidyverse.org

httr package: Wickham, H. (2023). httr: Tools for Working with URLs and HTTP. R package version 1.4.7. https://CRAN.R-project.org/package=httr

jsonlite package: Ooms, J. (2014). The jsonlite Package: A Practical and Consistent Mapping Between JSON Data and R Objects. arXiv:1403.2805 [stat.CO]. https://arxiv.org/abs/1403.2805

kableExtra package: Zhu, H. (2024). kableExtra: Construct Complex Table with 'kable' and Pipe Syntax. R package version 1.4.0. https://CRAN.R-project.org/package=kableExtra

knitr package: Xie, Y. (2023). knitr: A General-Purpose Package for Dynamic Report Generation in R. R package version 1.45. https://yihui.org/knitr/

lubridate package: Grolemund, G., & Wickham, H. (2011). Dates and Times Made Easy with lubridate. Journal of Statistical Software, 40(3), 1-25. https://www.jstatsoft.org/v40/i03/

newsanchor package: Ormerod, B., & Schoch, D. (2021). newsanchor: Client for the News API. R package version 0.1.1. https://CRAN.R-project.org/package=newsanchor

tempdisagg: Sax, C., & Steiner, P. (2013). Temporal Disaggregation of Time Series. The R Journal, 5(2), 80-87.

tidyquant package: Dancho, M., & Vaughan, D. (2026). tidyquant: Tidy Quantitative Financial Analysis. R package version 1.0.12. https://github.com/business-science/tidyquant

tidyverse package: Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., Takahashi, K., Vaughan, D., Wilke, C., Woo, K., & Yutani, H. (2019). Welcome to the Tidyverse. Journal of Open Source Software, 4(43), 1686. https://doi.org/10.21105/joss.01686

TTR package: Ulrich, J. (2026). TTR: Technical Trading Rules. R package version 0.24.4. https://github.com/joshuaulrich/ttr

zoo package: Zeileis, A., & Grothendieck, G. (2005). zoo: S3 Infrastructure for Regular and Irregular Time Series. Journal of Statistical Software, 14(6), 1-27. https://doi.org/10.18637/jss.v014.i06

Ocean Power Technologies SEC Filings: Ocean Power Technologies. (2026). SEC Filings. https://investors.oceanpowertechnologies.com/financial-information/sec-filings

News API: News API. (2026). News API. https://newsapi.org

Finnhub API: Finnhub. (2026). Finnhub Stock API. https://finnhub.io

Alpha Vantage API: Alpha Vantage. (2026). Alpha Vantage API. https://www.alphavantage.co